

Tribhuvan University

Faculty of Humanities and Social Sciences

Masters of Computer Application (MCA)

1st Semester

Advanced Database Management System(MCA-503)

Unit-5: NOSQL Database & Big Data Storage Systems 7hrs

Instructor

Tekendra Nath Yogi (M.SC CSIT TU)

Tekendranath@gmail.com

Contents

- **NOSQL Databases and Big Data Storage Systems [7 Hrs.]**
 - 5.1. Introduction to NOSQL Systems;
 - 5.2. The CAP Theorem;
 - 5.3. Document Based NOSQL Systems and MongoDB;
 - 5.4. NOSQL key-value storages;
 - 5.5. Column-based or wide column NOSQL Systems;
 - 5.6. NOSQL Graph Databases and Neo4j.

5.1 Introduction to NOSQL Systems

- **NOSQL**: Stands for Not only SQL.

- A Class of database systems developed to manage large amount of data.

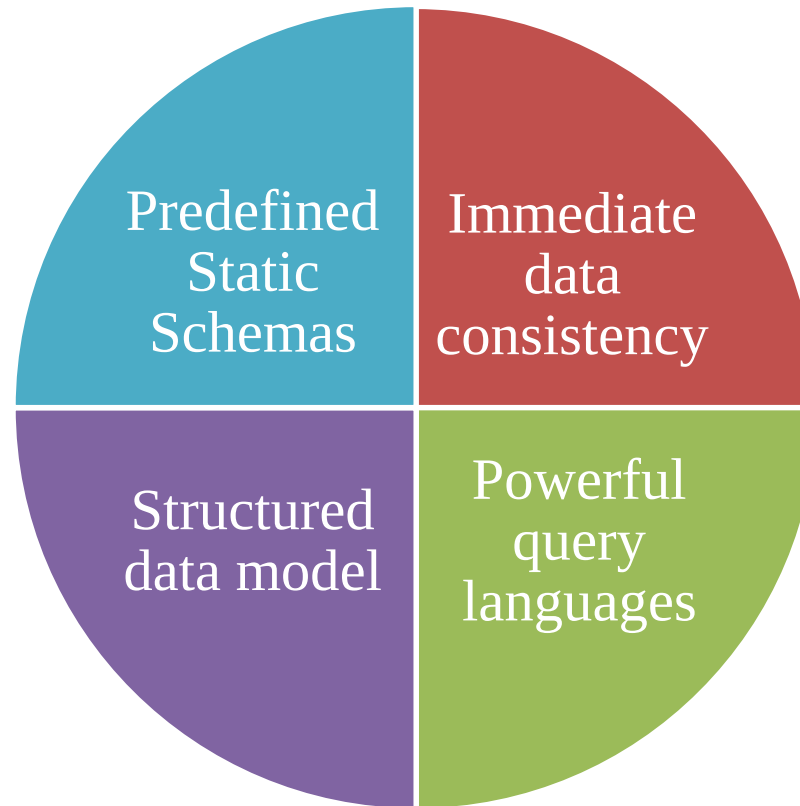
- *NOSQL systems are distributed databases with a focus on:*

- *Semi-structured data storage,*
- *High performance,*
- *Availability,*
- *Data replication, and*
- *Scalability*



Cont'd...

- **5.1.1. Emergence of NOSQL Systems:** Traditional relational SQL systems offer following features:



- These features make the traditional relational SQL systems more restrictive for larger organization using modern applications (not every applications requires these features).

Cont'd...

- Many International organization and companies such as Google, Amazon, Facebook, and Twitter are faced with applications that store vast amounts of data.
 - E-mail: Gmail by the google
 - Social media: Facebook, YouTube, etc.
 - Web links
 - User profiles
 - Marketing and sales
 - Posts and tweets, and
 - Road maps and spatial data

Cont'd...

- Some of the data for this type of application is not suitable for a traditional relational system and typically needs multiple types of databases and data storage systems.
 - i.e., Need systems other than traditional relational SQL systems to augment their data management needs.
- Some of the organizations that were faced with these data management and storage applications decided to develop their own systems.

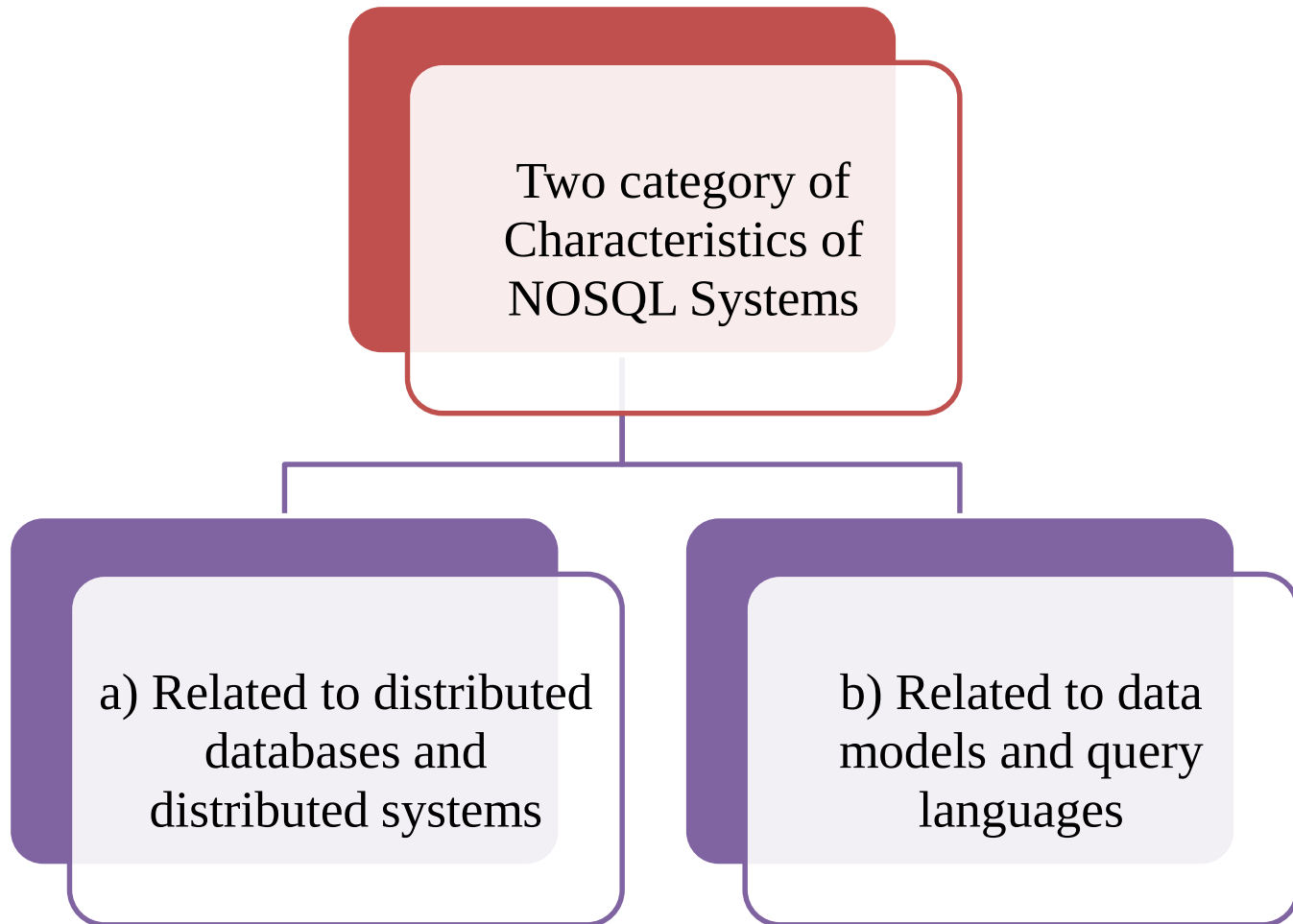
Cont'd...

- Few examples of NOSQL systems that have been developed:
 - **BigTable** (Google's proprietary) / Apache Hbase (Open source): Column-based or wide column store NOSQL system.
 - **DynamoDB** (Amazon): Key-value data store/key-tuple or key-object.
 - **Cassandra** (Facebook): Uses concepts from both key-value store and column-based systems.
 - **MongoDB** and **CouchDB**: Document stores
 - **Neo4J** and **GraphBase**: Graph-based NOSQL systems
 - **OrientDB**: Combines several concepts i.e., Hybrid NOSQL systems



Cont'd...

■ 5.1.2. Characteristics of NOSQL Systems :



Cont'd...

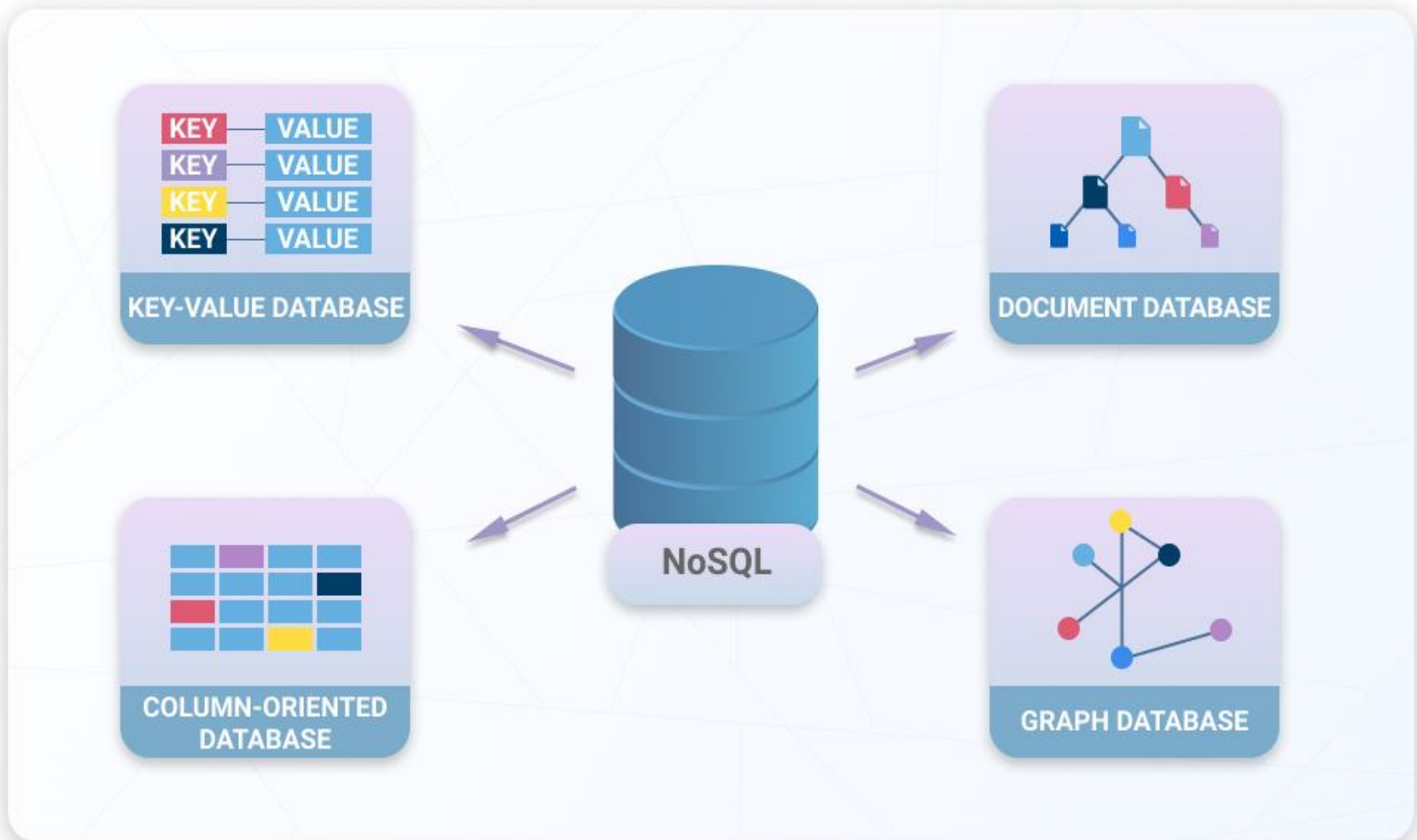
- **5.1.2. Characteristics of NOSQL Systems : a) Related to distributed databases and distributed systems:**
 - Scalability (horizontal and vertical): mainly horizontal scaling in NOSQL by adding more nodes for data storage and processing .
 - Availability, Replication, and Eventual Consistency
 - Replication models
 - Master-slave
 - Master-master
 - Sharding of files: Horizontal partitioning of the file records is often employed in NOSQL systems.
 - High performance data access: using hashing, partitioning, and indexing techniques instead of by using complex query conditions.

Cont'd...

- **5.1.2. Characteristics of NOSQL Systems : b) Related to data models and query languages:**
 - **Schema not required** as it supports the semi-structured, self describing data such as JSON, XML, etc. Also allow to specify the constraints on the data in the application programs.
 - **Less powerful query languages** than the traditional relational SQL systems.
 - **Versioning** of data items with the timestamps.

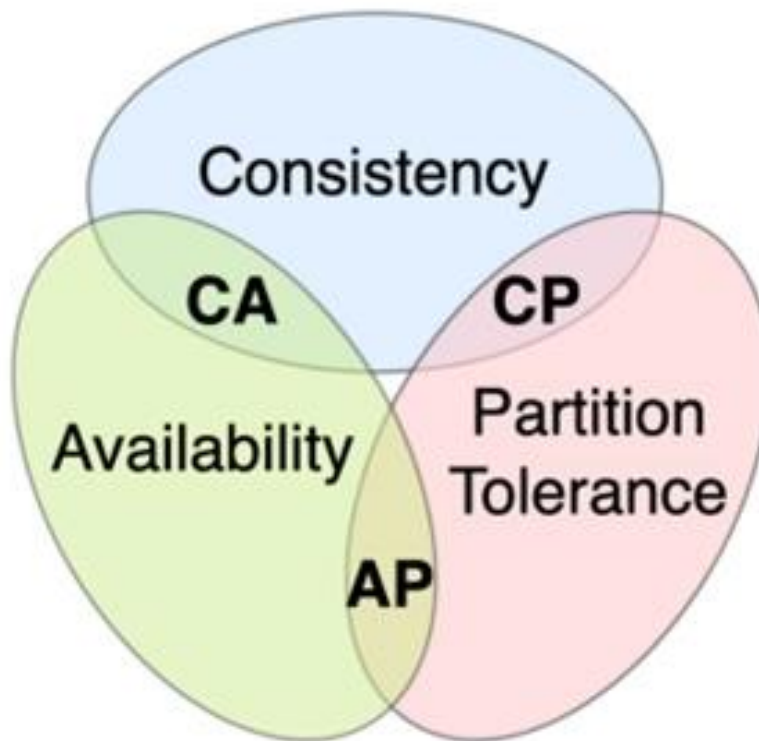
Cont'd...

■ 5.1.3. Categories of NOSQL systems:

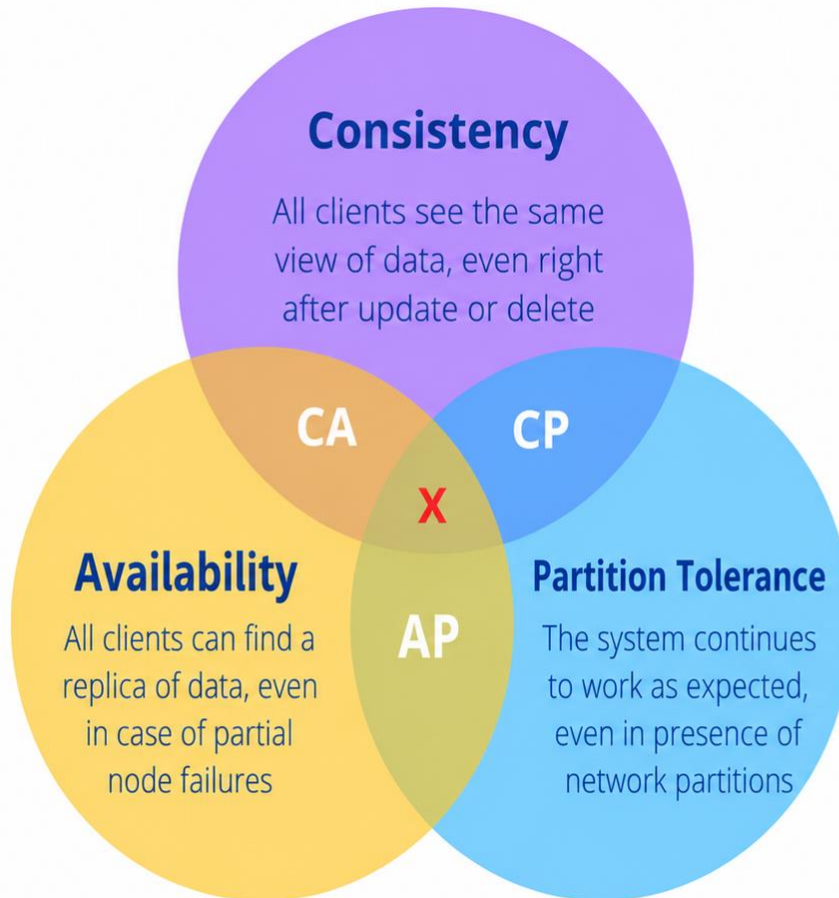


5.2 The CAP Theorem

- **CAP theorem:** The CAP theorem states that it is not possible to guarantee all three of the desirable properties—consistency, availability, and partition tolerance—at the same time in a distributed system with data replication.



Cont'd...



CA

sacrifices partition tolerance to ensure that all nodes in the system see the same data at the same time, while remaining available.

CP

sacrifices availability to ensure that all nodes in system see same data at same time, even in presence of a network partition.

AP

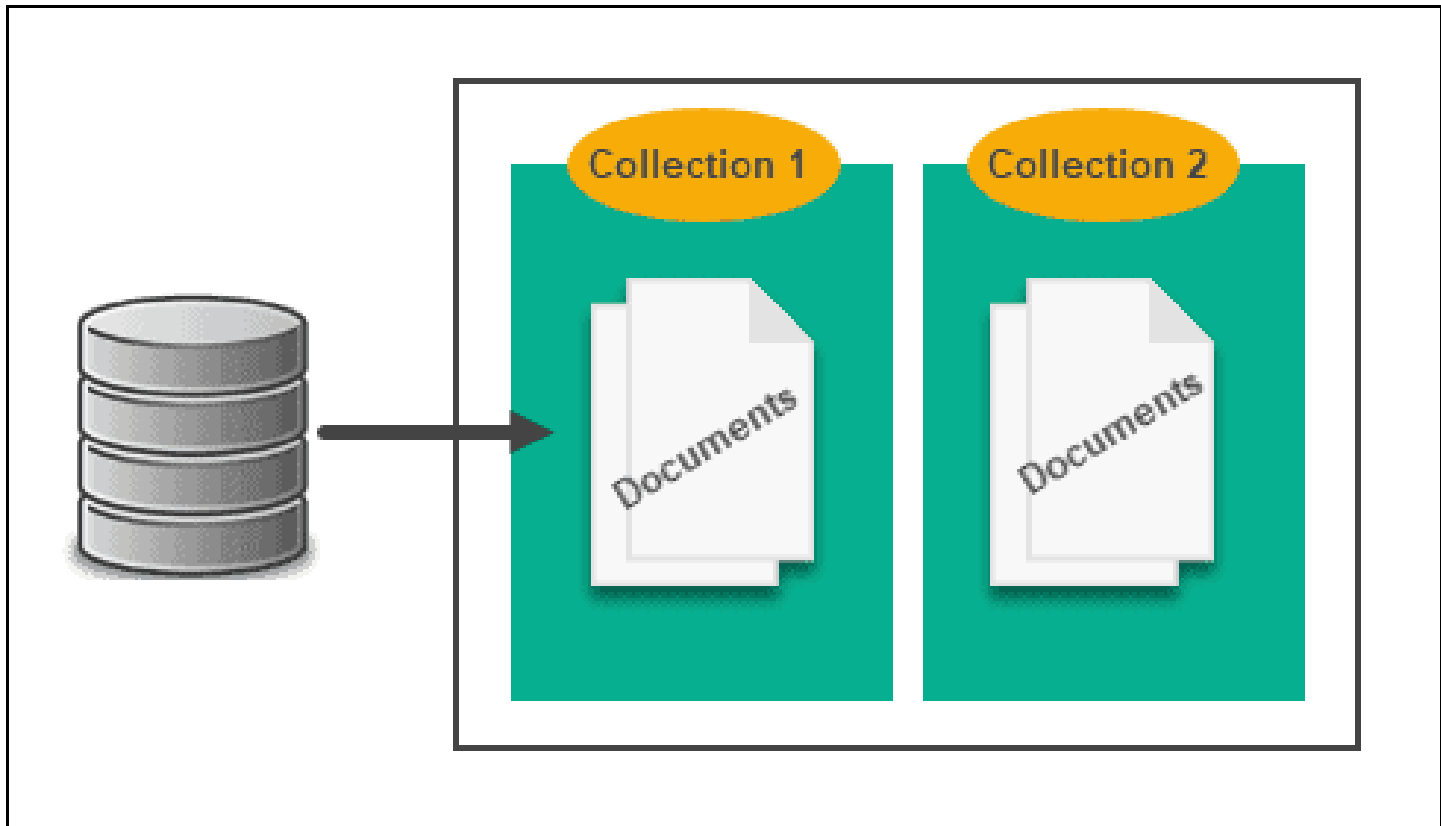
sacrifice consistency to ensure that all nodes in system are able to respond to requests, even in presence of a network partition.

Cont'd...

- Distributed system designer have to choose two of three to guarantee:
 - Weaker consistency level is often acceptable in NOSQL distributed data store
 - Guaranteeing availability and partition tolerance more important
 - Eventual consistency often adopted i.e., All replicas will gradually become consistent in the absence of updates

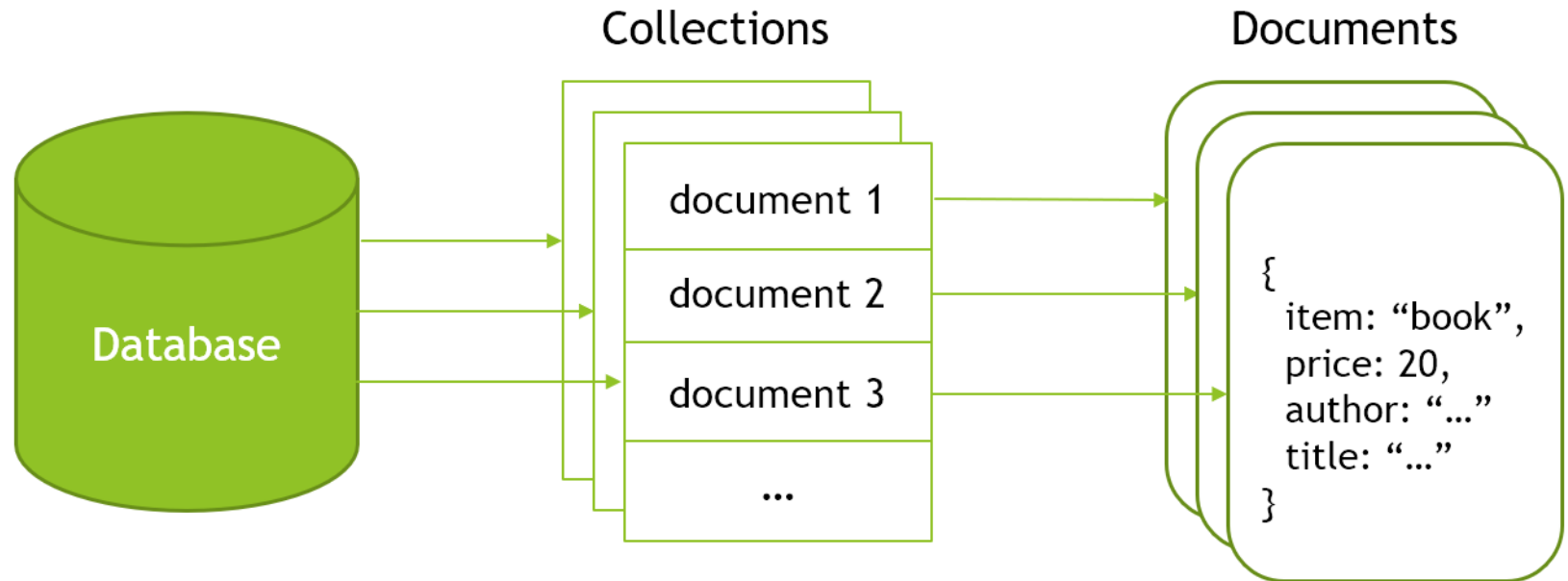
5.3 Document-Based NOSQL Systems and MongoDB

- Document-based NOSQL systems (Document stores) store data as collections of similar documents.



Cont'd...

- The documents are specified as self-describing data hence, there is no need to specify a schema.



Cont'd...

- The documents in a collection should be similar, but they can have different data elements (attributes).
- New documents can have new data elements that do not exist in any of the current documents in the collection.

Document 1

```
{
  "id": "1",
  "name": "John Smith",
  "isActive": true,
  "dob": "1964-30-08"
}
```

Document 2

```
{
  "id": "2",
  "fullName": "Sarah Jones",
  "isActive": false,
  "dob": "2002-02-18"
}
```

Document 3

```
{
  "id": "3",
  "fullName": {
    "first": "Adam",
    "last": "Stark"
  },
  "isActive": true,
  "dob": "2015-04-19"
}
```

Cont'd...

- Documents can be specified in various format: XML or JSON

```
{
  "books": [
    {
      "id": "01",
      "language": "Java",
      "author": "H. Javeson",
      "year": 2015
    },
    {
      "id": "07",
      "language": "C++",
      "edition": "second",
      "author": "E. Sepp",
      "price": 22.25
    }
  ]
}
```

```
<?xml version="1.0"?>
<books>
  <book>
    <id>01</id>
    <language>Java</language>
    <author>H. Javeson</author>
    <year>2015</year>
  </book>
  <book>
    <id>07</id>
    <language>C++</language>
    <edition>second</edition>
    <author>E. Sepp</author>
    <price>22.25</price>
  </book>
</books>
```

JSON is most common in NOSQL DB.

Cont'd...

- Example of document-based NOSQL systems: MongoDB, CouchDB, etc.



mongoDB



CouchDB
relax

Cont'd...

■ 5.3.1. MongoDB Data Model:

- Documents stored in binary JSON (BSON) format
 - Individual documents stored in a collection
 - Example command
 - First parameter specifies name of the collection
 - Collection options include limits on size and number of documents
- db.createCollection(“project”, { capped : true, size : 1310720, max : 500 })**
- Each document in collection has unique ObjectID field called `_id`

Cont'd...

- A collection does not have a schema
 - Structure of the data fields in documents chosen based on how documents will be accessed
 - User can choose normalized or denormalized design

Cont'd...

■ 5.3.2. MongoDB CRUD Operations:

- Document creation using insert operation

db.<collection_name>.insert(<document(s)>)

- Document deletion using remove operation

db.<collection_name>.remove(<condition>)

- Document reading using find operation

db.<collection_name>.find(<condition>)

Cont'd...

- Example of simple documents in MongoDB: (a) Denormalized document design with embedded subdocuments

(a) project document with an array of embedded workers:

```
{
  _id: "P1",
  Pname: "ProductX",
  Plocation: "Bellaire",
  Workers:
  [
    {
      Ename: "John Smith",
      Hours: 32.5
    },
    {
      Ename: "Joyce English",
      Hours: 20.0
    }
  ]
};
```

Cont'd...

- Example of simple documents in MongoDB: (b) Embedded array of document references

(b) project document with an embedded array of worker ids:

```
{
  _id: "P1",
  Pname: "ProductX",
  Plocation: "Bellaire",
  WorkerIds: [ "W1", "W2" ]
}
{
  _id: "W1",
  Ename: "John Smith",
  Hours: 32.5
}
{
  _id: "W2",
  Ename: "Joyce English",
  Hours: 20.0
}
```

Cont'd...

- **Example of simple documents in MongoDB (c) Normalized documents**

(c) normalized project and worker documents (not a fully normalized design for M:N relationships):

```
{
  _id: "P1",
  Pname: "ProductX",
  Plocation: "Bellaire"
}
{
  _id: "W1",
  Ename: "John Smith",
  ProjectId: "P1",
  Hours: 32.5
}
{
  _id: "W2",
  Ename: "Joyce English",
  ProjectId: "P1",
  Hours: 20.0
}
```

Cont'd...

- Example of simple documents in MongoDB (d) Inserting the documents in (c) into their collections

(d) inserting the documents in (c) into their collections “project” and “worker”:

```
db.project.insert( { _id: “P1”, Pname: “ProductX”, Plocation: “Bellaire” } )
```

```
db.worker.insert
```

```
(
```

```
[
```

```
    { _id: “W1”, Ename: “John Smith”, ProjectId: “P1”, Hours: 32.5 },
```

```
    { _id: “W2”, Ename: “Joyce English”, ProjectId: “P1”, Hours: 20.0 }
```

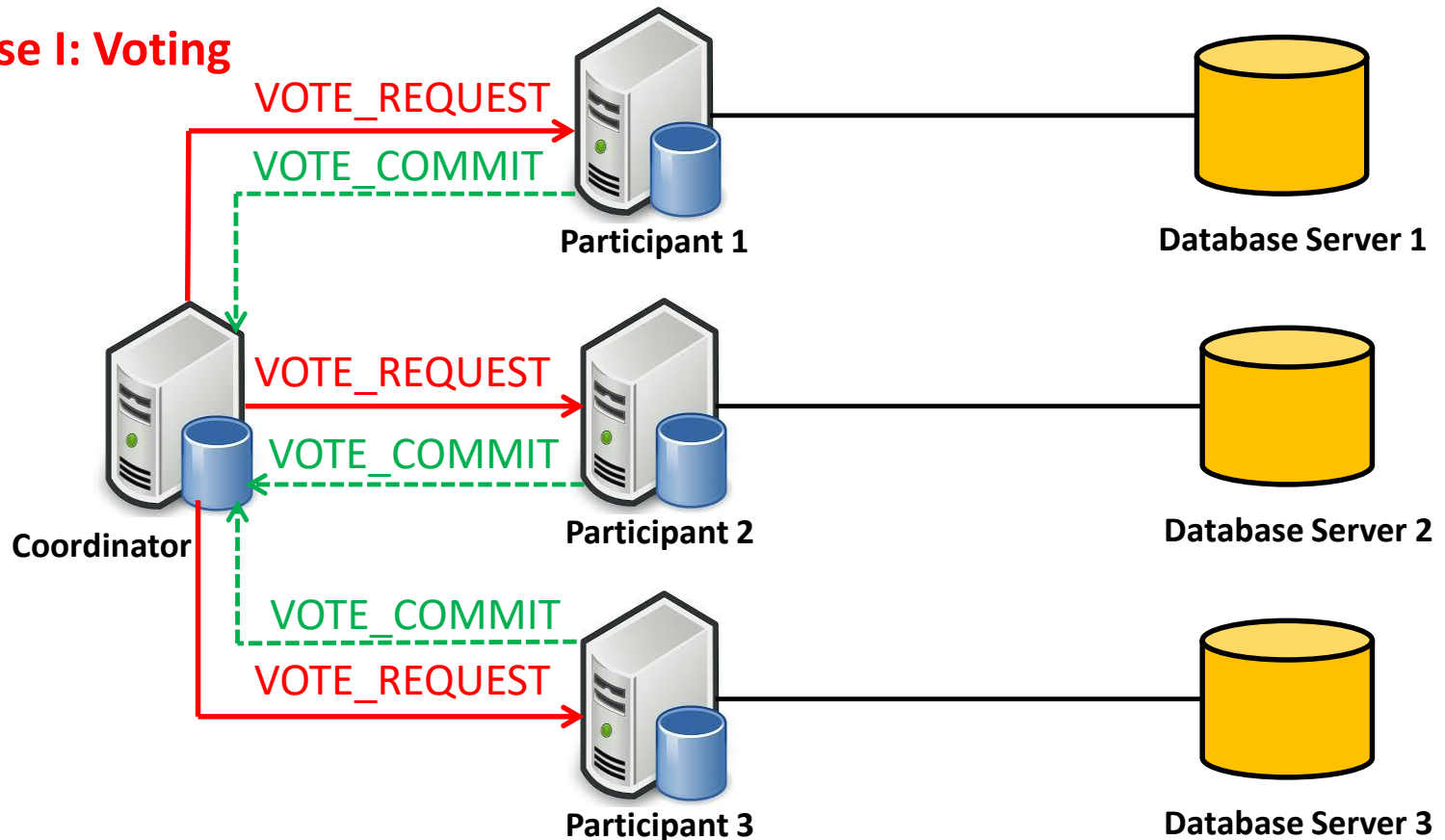
```
]
```

```
)
```

Cont'd...

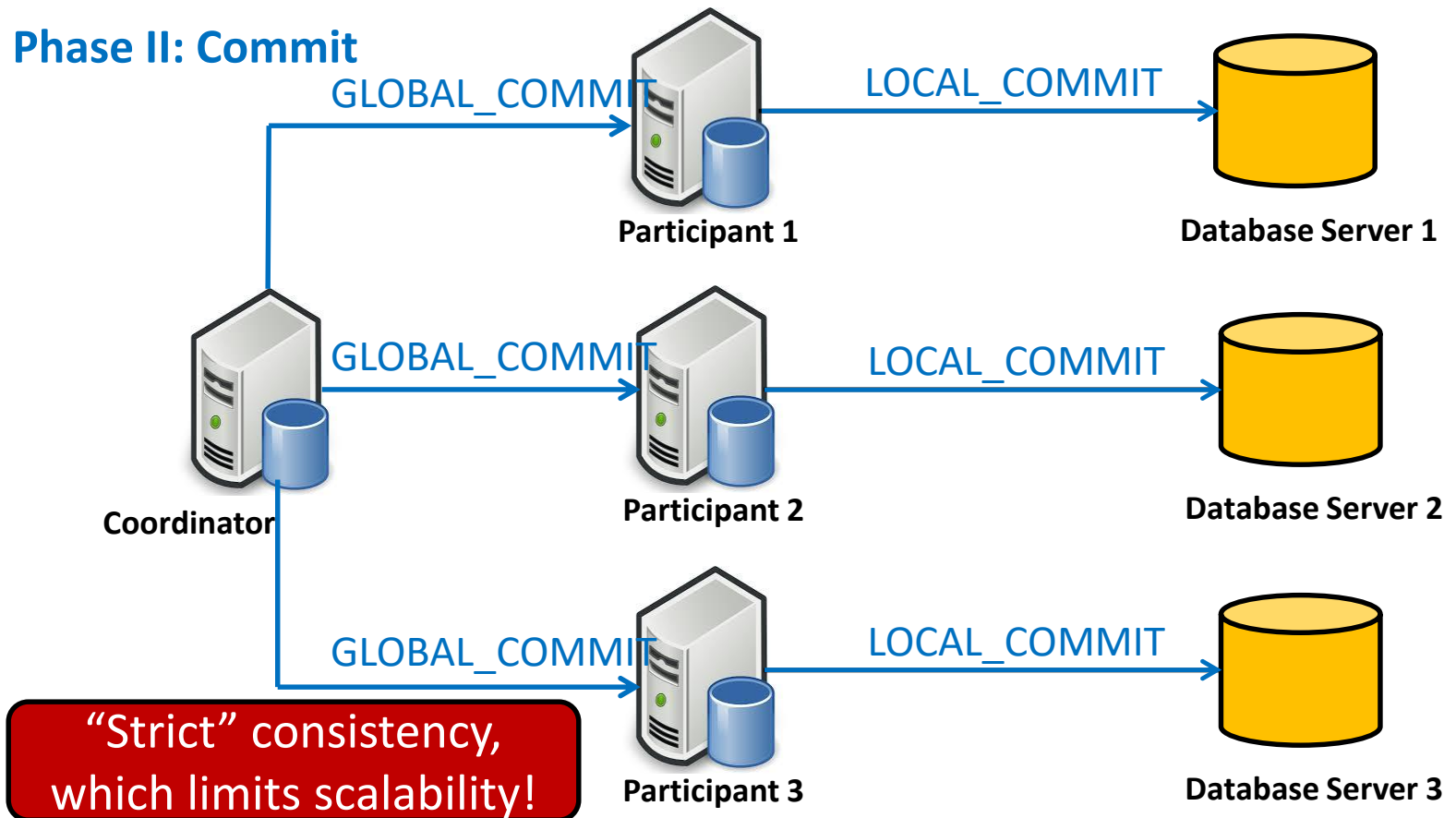
- **3.2.3. MongoDB Distributed Systems Characteristics:** Two-phase commit method - Used to ensure atomicity and consistency of multi-document transactions.

Phase I: Voting



Cont'd...

- 3.2.3. MongoDB Distributed Systems Characteristics: Two-phase commit method



Cont'd...

- **3.2.3. MongoDB Distributed Systems Characteristics:** Replication in MongoDB
 - Concept of replica set to create multiple copies on different nodes
 - Variation of master-slave approach
 - Primary copy, secondary copy, and arbiter
 - Arbiter participates in elections to select new primary if needed
 - All write operations applied to the primary copy and propagated to the secondaries
 - User can choose read preference
 - Read requests can be processed at any replica

Cont'd...

- **3.2.3. MongoDB Distributed Systems Characteristics:** Sharding in MongoDB
 - Horizontal partitioning divides the documents into disjoint partitions (shards)
 - Allows adding more nodes as needed
 - Shards stored on different nodes to achieve load balancing
 - Partitioning field (shard key) must exist in every document in the collection
 - Must have an index
 - Range partitioning
 - Creates chunks by specifying a range of key values
 - Works best with range queries
 - Hash partitioning
 - Partitioning based on the hash values of each shard key

5.4 NOSQL Key-Value Stores

- **Key-value** stores focus on high performance, availability, and scalability
 - Can store structured, unstructured, or semi-structured data
- **Key:** unique identifier associated with a data item
 - Used for fast retrieval
- **Value:** the data item itself
 - Can be string or array of bytes
 - Application interprets the structure
- No query language
- Examples: DynamoDB, Voldemort, etc.



Cont'd...

- **5.4.1. DynamoDB Overview:** DynamoDB part of Amazon's Web Services/SDK platforms
 - Proprietary
- Table holds a collection of self-describing items
- Item consists of attribute-value pairs
 - Attribute values can be single or multi-valued
- Primary key used to locate items within a table
 - Can be single attribute or pair of attributes

Cont'd...

- **5.4.2. Voldemort Key-Value Distributed Data Store:**
- Voldemort: open source key-value system similar to DynamoDB
- Voldemort features
 - Simple basic operations (get, put, and delete)
 - High-level formatted data values
 - Consistent hashing for distributing (key, value) pairs
 - Consistency and versioning
 - Concurrent writes allowed
 - Each write associated with a vector clock

Cont'd...

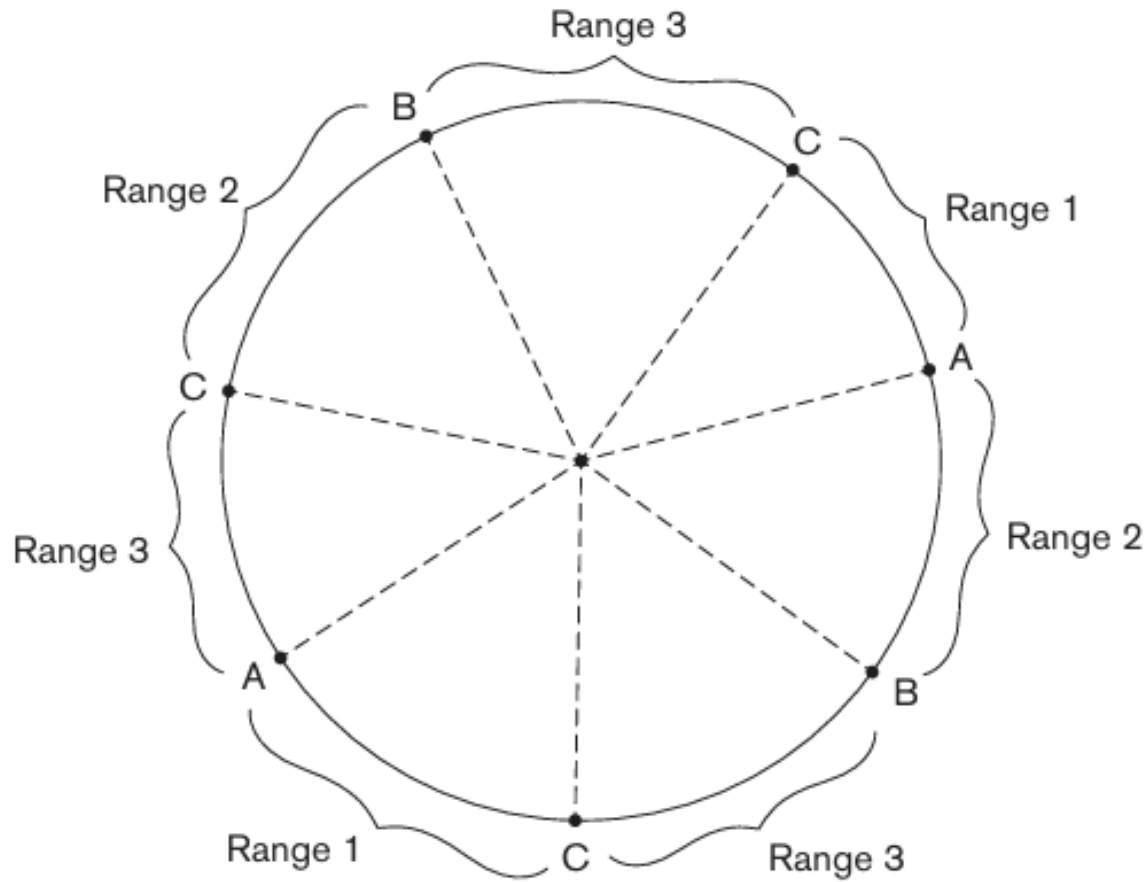


Figure: Example of consistent hashing (a) Ring having three nodes A, B, and C, with C having greater capacity. The $h(K)$ values that map to the circle points in *range 1* have their (k, v) items stored in node A, *range 2* in node B, *range 3* in node C

Cont'd...

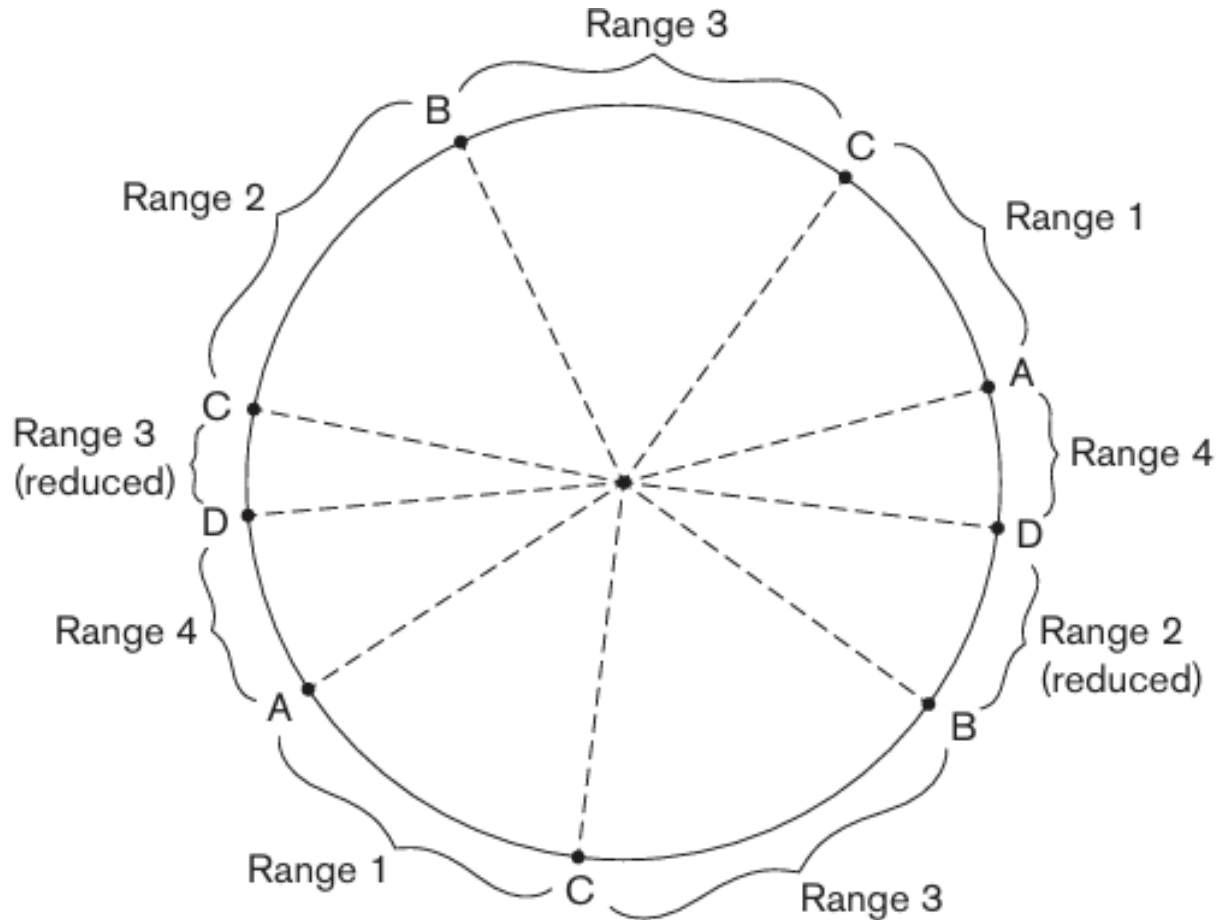


Figure: Example of consistent hashing (b) Adding a node D to the ring. Items in *range 4* are moved to the node D from node B (*range 2* is reduced) and node C (*range 3* is reduced)

Cont'd...

- **5.4.2. Examples of Other Key-Value Stores:**
- **Oracle** key-value store
 - Oracle NOSQL Database
- **Redis** key-value cache and store
 - Caches data in main memory to improve performance
 - Offers master-slave replication and high availability
 - Offers persistence by backing up cache to disk
- **Apache** Cassandra
 - Offers features from several NOSQL categories
 - Used by Facebook and others

5.5 Column-Based or Wide Column NOSQL Systems

- **BigTable:** Google's distributed storage system for big data
 - Used in Gmail
 - Uses Google File System for data storage and distribution
- **Apache Hbase:** a similar, open source system
 - Uses Hadoop Distributed File System (HDFS) for data storage
 - Can also use Amazon's Simple Storage System (S3)

Cont'd...

- **5.5.1. Hbase Data Model and Versioning:** Data organization concepts
 - Namespaces
 - Tables
 - Column families
 - Column qualifiers
 - Columns
 - Rows
 - Data cells
- Data is self-describing

Cont'd...

- HBase stores multiple versions of data items
 - Timestamp associated with each version
- Each row in a table has a unique row key
- Table associated with one or more column families
- Column qualifiers can be dynamically specified as new table rows are created and inserted
- Namespace is collection of tables
- Cell holds a basic data item

Cont'd...

- creating a table: create 'EMPLOYEE', 'Name', 'Address', 'Details'

Cont'd...

- (b) inserting some row data in the EMPLOYEE table:

```
put 'EMPLOYEE', 'row1', 'Name:Fname', 'John'
put 'EMPLOYEE', 'row1', 'Name:Lname', 'Smith'
put 'EMPLOYEE', 'row1', 'Name:Nickname', 'Johnny'
put 'EMPLOYEE', 'row1', 'Details:Job', 'Engineer'
put 'EMPLOYEE', 'row1', 'Details:Review', 'Good'
put 'EMPLOYEE', 'row2', 'Name:Fname', 'Alicia'
put 'EMPLOYEE', 'row2', 'Name:Lname', 'Zelaya'
put 'EMPLOYEE', 'row2', 'Name:MName', 'Jennifer'
put 'EMPLOYEE', 'row2', 'Details:Job', 'DBA'
put 'EMPLOYEE', 'row2', 'Details:Supervisor', 'James Borg'
put 'EMPLOYEE', 'row3', 'Name:Fname', 'James'
put 'EMPLOYEE', 'row3', 'Name:Minit', 'E'
put 'EMPLOYEE', 'row3', 'Name:Lname', 'Borg'
put 'EMPLOYEE', 'row3', 'Name:Suffix', 'Jr.'
put 'EMPLOYEE', 'row3', 'Details:Job', 'CEO'
put 'EMPLOYEE', 'row3', 'Details:Salary', '1,000,000'
```

Cont'd...

- (c) Some Hbase basic CRUD operations:
 - Creating a table:
 - `create <tablename>, <column family>, <column family>, ...`
 - Inserting Data:
 - `put <tablename>, <rowid>, <column family>:<column qualifier>, <value>`
 - Reading Data (all data in a table):
 - `scan <tablename>`
 - Retrieve Data (one item):
 - `get <tablename>,<rowid>`

Cont'd...

■ 5.5.2. Hbase Crud Operations:

- Provides only low-level CRUD (create, read, update, delete) operations
- Application programs implement more complex operations
- Create
 - Creates a new table and specifies one or more column families associated with the table
- Put
 - Inserts new data or new versions of existing data items
- Get
 - Retrieves data associated with a single row
- Scan
 - Retrieves all the rows

Cont'd...

- **5.5.3. Hbase Storage and Distributed System Concepts**
- Each Hbase table divided into several regions
 - Each region holds a range of the row keys in the table
 - Row keys must be lexicographically ordered
 - Each region has several stores
 - Column families are assigned to stores
- Regions assigned to region servers for storage
 - Master server responsible for monitoring the region servers
- Hbase uses Apache Zookeeper and HDFS

5.6. NOSQL Graph Databases and Neo4j

- **Graph databases:**

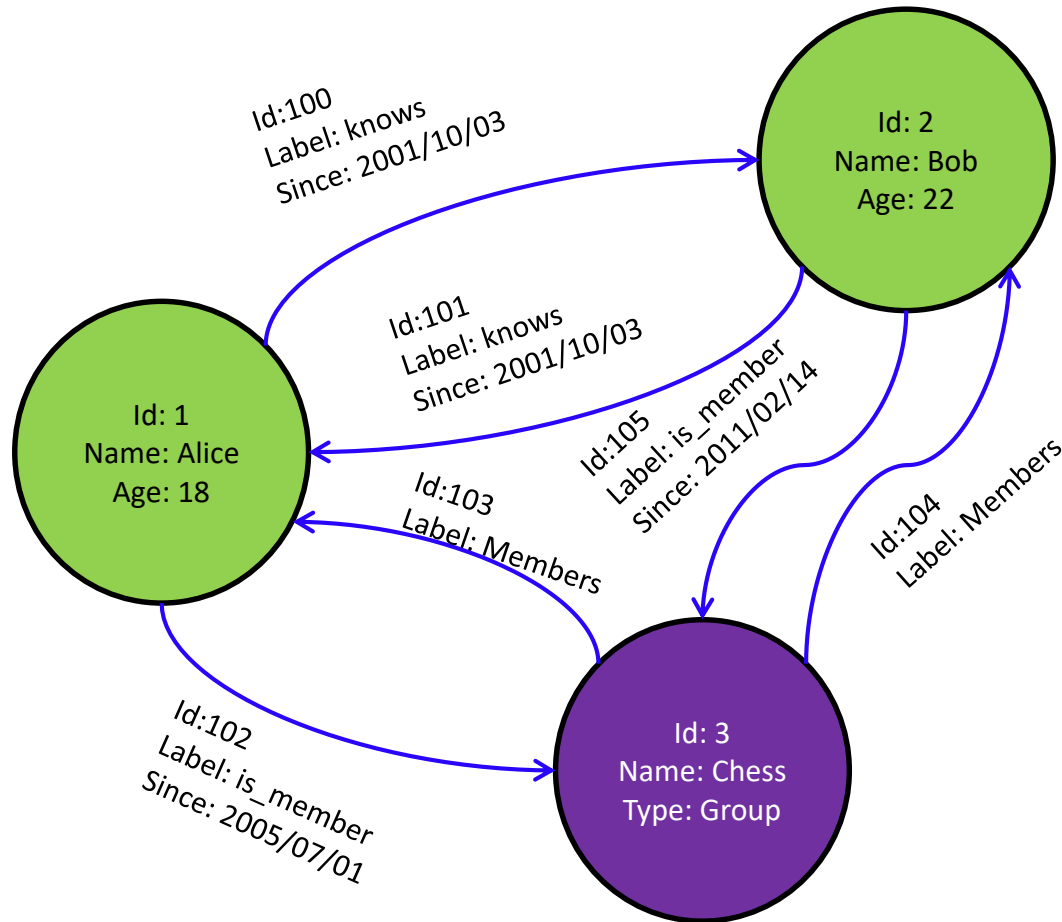
- In graph data bases data is represented as a graph.
- Graph is a collection of vertices (nodes) and edges.
- Both nodes and edges can be labeled to indicate the types of entities and relationships they represent and
- It is possible to store data associated with both individual nodes and individual edges.
- Well known example of graph database is Neo4j which is **Open source system.**

Cont'd...

- **5.6.1. Neo4j Data Model:**
 - Uses concepts of nodes and relationships
 - Nodes can have labels: Zero, one, or several
 - Both nodes and relationships can have properties
 - Each relationship has a start node, end node, and a relationship type
 - Properties specified using a map pattern

Cont'd...

■ Example,



Cont'd...

- Creating nodes in Neo4j
 - CREATE command
 - Part of high-level declarative query language Cypher
 - Node label can be specified when node is created
 - Properties are enclosed in curly brackets

Cont'd...

- Examples in Neo4j using the Cypher language (a) Creating some nodes

```
CREATE (e1: EMPLOYEE, {Empid: '1', Lname: 'Smith', Fname: 'John', Minit: 'B'})
```

```
CREATE (e2: EMPLOYEE, {Empid: '2', Lname: 'Wong', Fname: 'Franklin'})
```

```
CREATE (e3: EMPLOYEE, {Empid: '3', Lname: 'Zelaya', Fname: 'Alicia'})
```

```
CREATE (e4: EMPLOYEE, {Empid: '4', Lname: 'Wallace', Fname: 'Jennifer', Minit: 'S'})
```

```
...
```

```
CREATE (d1: DEPARTMENT, {Dno: '5', Dname: 'Research'})
```

```
CREATE (d2: DEPARTMENT, {Dno: '4', Dname: 'Administration'})
```

```
...
```

```
CREATE (p1: PROJECT, {Pno: '1', Pname: 'ProductX'})
```

```
CREATE (p2: PROJECT, {Pno: '2', Pname: 'ProductY'})
```

```
CREATE (p3: PROJECT, {Pno: '10', Pname: 'Computerization'})
```

```
CREATE (p4: PROJECT, {Pno: '20', Pname: 'Reorganization'})
```

```
...
```

```
CREATE (loc1: LOCATION, {Lname: 'Houston'})
```

```
CREATE (loc2: LOCATION, {Lname: 'Stafford'})
```

```
CREATE (loc3: LOCATION, {Lname: 'Bellaire'})
```

```
CREATE (loc4: LOCATION, {Lname: 'Sugarland'})
```

Cont'd...

- Examples in Neo4j using the Cypher language (b) Creating some relationships

creating some relationships for the COMPANY data :

```
CREATE (e1) – [ : WorksFor ] –> (d1)
```

```
CREATE (e3) – [ : WorksFor ] –> (d2)
```

...

```
CREATE (d1) – [ : Manager ] –> (e2)
```

```
CREATE (d2) – [ : Manager ] –> (e4)
```

...

```
CREATE (d1) – [ : LocatedIn ] –> (loc1)
```

```
CREATE (d1) – [ : LocatedIn ] –> (loc3)
```

```
CREATE (d1) – [ : LocatedIn ] –> (loc4)
```

```
CREATE (d2) – [ : LocatedIn ] –> (loc2)
```

...

```
CREATE (e1) – [ : WorksOn, {Hours: ‘32.5’} ] –> (p1)
```

```
CREATE (e1) – [ : WorksOn, {Hours: ‘7.5’} ] –> (p2)
```

```
CREATE (e2) – [ : WorksOn, {Hours: ‘10.0’} ] –> (p1)
```

```
CREATE (e2) – [ : WorksOn, {Hours: 10.0} ] –> (p2)
```

```
CREATE (e2) – [ : WorksOn, {Hours: ‘10.0’} ] –> (p3)
```

```
CREATE (e2) – [ : WorksOn, {Hours: 10.0} ] –> (p4)
```

Cont'd...

- Path
 - Traversal of part of the graph
 - Typically used as part of a query to specify a pattern
- Schema optional in Neo4j
- Indexing and node identifiers
 - Users can create for the collection of nodes that have a particular label
 - One or more properties can be indexed

Cont'd...

- **5.6.2. The Cypher Query Language of Neo4j:**
 - Cypher query made up of clauses
 - Result from one clause can be the input to the next clause in the query

Cont'd...

- Examples in Neo4j using the Cypher language (c) Basic syntax of Cypher queries (Basic simplified syntax of some common Cypher clauses):
 - Finding nodes and relationships that match a pattern: MATCH <pattern>
 - Specifying aggregates and other query variables: WITH <specifications>
 - Specifying conditions on the data to be retrieved: WHERE <condition>
 - Specifying the data to be returned: RETURN <data>
 - Ordering the data to be returned: ORDER BY <data>
 - Limiting the number of returned data items: LIMIT <max number>
 - Creating nodes: CREATE <node, optional labels and properties>
 - Creating relationships: CREATE <relationship, relationship type and optional properties>
 - Deletion: DELETE <nodes or relationships>
 - Specifying property values and labels: SET <property values and labels>
 - Removing property values and labels: REMOVE <property values and labels>

Cont'd...

- Examples in Neo4j using the Cypher language (d) Examples of Cypher queries

```
1. MATCH (d : DEPARTMENT {Dno: '5'}) – [ : LocatedIn ] → (loc)  
RETURN d.Dname , loc.Lname
```

```
2. MATCH (e: EMPLOYEE {Empid: '2'}) – [ w: WorksOn ] → (p)  
RETURN e.Ename , w.Hours, p.Pname
```

```
3. MATCH (e ) – [ w: WorksOn ] → (p: PROJECT {Pno: 2})  
RETURN p.Pname, e.Ename , w.Hours
```

```
4. MATCH (e) – [ w: WorksOn ] → (p)  
RETURN e.Ename , w.Hours, p.Pname  
ORDER BY e.Ename
```

Cont'd...

- Examples in Neo4j using the Cypher language (d) Examples of Cypher queries

```
5. MATCH (e) – [ w: WorksOn ] → (p)
RETURN e.Ename , w.Hours, p.Pname
ORDER BY e.Ename
LIMIT 10
```

```
6. MATCH (e) – [ w: WorksOn ] → (p)
WITH e, COUNT(p) AS numOfprojs
WHERE numOfprojs > 2
RETURN e.Ename , numOfprojs
ORDER BY numOfprojs
```

```
7. MATCH (e) – [ w: WorksOn ] → (p)
RETURN e , w, p
ORDER BY e.Ename
LIMIT 10
```

```
8. MATCH (e: EMPLOYEE {Empid: '2'})
SET e.Job = 'Engineer'
```

Cont'd...

- **5.6.3. Neo4j Interfaces and Distributed System Characteristics:**
- Enterprise edition versus community edition
 - Enterprise edition supports caching, clustering of data, and locking
- Graph visualization interface
 - Subset of nodes and edges in a database graph can be displayed as a graph
 - Used to visualize query results
- Master-slave replication
- Caching
- Logical logs

Thank You !

